Abstract – Blood Donation System

Blood is a vital resource in healthcare used for surgeries, accident treatments, cancer therapy, organ transplants, and maternity care. Since blood cannot be artificially produced, hospitals depend on voluntary donors and properly maintained blood banks. However, traditional blood management systems are often manual and time-consuming, which can create difficulties in locating blood quickly during emergency situations.

The Blood Donation System is an Android-based application designed to simplify and organize the process of blood donation and blood distribution. The system connects donors, blood banks, and patients on a single digital platform. Donors can register their personal details, blood group, and availability status. Blood banks can manage and update blood stock information, while patients or their relatives can easily search for required blood groups and send requests through the application.

The main objective of this system is to improve communication between donors, blood banks, and patients, and to reduce the time required to locate blood in emergencies. By using mobile technology and a centralized database, the application provides quick access to blood availability information anytime and anywhere. The system also promotes awareness about voluntary blood donation and encourages more people to contribute to saving lives.

CHAPTER – I

INTRODUCTION

Introduction – Blood Donation System

Blood is one of the most important and life-saving resources in modern healthcare. It plays a crucial role in many medical treatments such as surgeries, accident recovery, cancer therapy, organ transplants, and maternity care. Every day, hospitals and medical centers require a large amount of blood to treat patients and save lives. Since blood cannot be manufactured artificially in laboratories, the only source of blood is voluntary donors. Therefore, maintaining an adequate supply of blood and managing blood banks effectively is very important for the healthcare system.

In many hospitals and blood banks, the process of managing blood donation and blood distribution is still carried out using traditional or manual methods. These methods include maintaining records in registers, contacting donors through phone calls, and manually checking the availability of blood groups. Such methods are often time-consuming and may lead to delays in emergency situations. When patients urgently require blood, their family members usually need to visit multiple hospitals or blood banks to check availability, which can cause stress, confusion, and loss of valuable time.

Another challenge in the current system is the lack of proper communication between blood donors, blood banks, and patients. Many people are willing to donate blood, but they may not know where and when their donation is needed. Similarly, blood banks may not always have updated information about available donors, and patients may struggle to find the required blood group quickly. These issues highlight the need for a more organized, efficient, and technology-based system to manage blood donation activities.

With the rapid development of mobile technology and digital communication, it has become easier to create applications that can connect people and provide services quickly. Smartphones are widely used by people for various daily activities such as communication, information sharing, and accessing online services. Using this technology, a digital solution can be developed to improve the blood donation process and make it more efficient.

The **Blood Donation System** is an Android-based application designed to simplify and manage the process of blood donation and blood distribution. This system acts as a bridge between blood donors, blood banks, and patients who require blood. Through this application, donors can easily register their personal information, blood group, contact details, and availability status. This information is stored in a digital database and can be accessed whenever required.

Blood banks can use the system to manage and update their blood stock details in real time. They can record the available blood groups, track donations, and maintain proper information about blood storage. This helps in ensuring that the blood bank always has accurate information about the available blood units.

Patients or their relatives who need blood can use the application to search for the required blood group and find nearby blood banks or available donors. They can send requests through the system, which helps in reducing the time needed to locate blood during emergency situations. This quick access to information can play a vital role in saving lives.

The Blood Donation System not only helps in managing blood availability but also encourages people to participate in voluntary blood donation. The application can spread awareness about the importance of blood donation and motivate more individuals to become donors. Regular donors can update their availability and contribute to helping patients in need.

In addition, the system improves communication and coordination among all users. Donors, blood banks, and patients are connected through a centralized digital platform, which makes the process more transparent and efficient. The use of a digital database also helps in maintaining accurate records and reduces the chances of data loss compared to manual record-keeping systems.

Overall, the Blood Donation System is designed to provide a reliable, fast, and user-friendly platform for managing blood donation activities. By using modern mobile technology, the system helps reduce delays, improves accessibility to blood information, and supports hospitals and blood banks in providing better healthcare services. Most importantly, it contributes to saving human lives by ensuring that blood is available when it is needed the most.

CHAPTER – II

SYSTEM STUDY AND ANALYSIS

2.1 PROBLEM STATEMENT

Blood is a critical resource in healthcare and is required for various medical treatments such as surgeries, accident recovery, cancer therapy, and maternity care. Hospitals and medical centers must maintain an adequate supply of different blood groups to handle both routine and emergency medical situations. However, managing blood availability and locating the required blood group at the right time remains a major challenge in many healthcare systems.

In many places, blood bank management and donor information are still maintained using manual methods such as paper records, phone calls, and physical visits. These traditional systems are often inefficient, time-consuming, and prone to errors. During emergency situations, patients or their relatives may face serious difficulties in finding the required blood group quickly. They often need to contact multiple blood banks or search for donors through personal networks, which can delay treatment and increase the risk to patients' lives.

Another major issue is the lack of proper coordination and communication between blood donors, blood banks, and patients. Many potential donors are willing to donate blood, but there is no centralized system to connect them with people who need blood. Blood banks may also face challenges in maintaining accurate and updated records of blood stock and donor availability.

Because of these problems, there is a strong need for a reliable and efficient system that can manage blood donor information, track blood availability, and help patients quickly locate the required blood group. The **Blood Donation System** aims to solve these issues by providing a digital platform that connects donors, blood banks, and patients, making the process of blood donation and blood distribution faster, easier, and more organized.

2.2 EXISTING SYSTEM

In the existing system, the management of blood donation and blood distribution is mainly carried out using traditional and manual methods. Blood banks usually maintain donor information, blood group details, and blood stock records in registers or basic computer files. These records are often updated manually, which can be time-consuming and may lead to errors or outdated information.

When a patient requires blood, their family members usually have to visit different hospitals or blood banks to check whether the required blood group is available. In many cases, they also depend on phone calls, personal contacts, or social media to find donors. This process takes a significant amount of time, especially during emergency situations where immediate blood transfusion is required.

Another limitation of the existing system is the lack of proper coordination between donors, blood banks, and patients. Many people are willing to donate blood, but there is no proper platform that connects donors directly with those who need blood. As a result, blood banks may face shortages of certain blood groups even when potential donors are available.

In addition, maintaining records manually makes it difficult to update blood stock information in real time. Sometimes, the available blood information may not be accurate, which can cause confusion and delays in providing blood to patients. The absence of a centralized system also makes it difficult to track donors, manage requests, and maintain proper communication between all participants.

Because of these limitations, the existing system is not efficient enough to handle emergency blood requirements quickly and effectively. This highlights the need for a modern and automated system that can simplify the blood donation process and provide quick access to blood availability information.

2.2.1 DISADVANTAGES OF EXISTING SYSTEM

1. **Time-Consuming Process**
   Finding the required blood group takes a lot of time because patients or their relatives must visit multiple hospitals or blood banks.
2. **Manual Record Maintenance**
   Blood donor details and blood stock information are maintained manually, which can lead to errors and data loss.
3. **Lack of Real-Time Updates**
   Blood availability is not updated instantly, so the information may be outdated or inaccurate.
4. **Poor Communication**
   There is no proper system to connect donors, blood banks, and patients efficiently.
5. **Difficulty in Emergency Situations**
   During emergencies, it becomes very difficult to quickly find the required blood group.
6. **Limited Donor Reach**
   Many potential donors are willing to donate blood, but they are not easily reachable through the existing system.
7. **Inefficient Data Management**
   Managing and searching donor or blood stock data in manual records is difficult and slow.
8. **No Centralized System**
   Information about donors and blood availability is scattered across different hospitals and blood banks.

2.3 PROPOSED SYSTEM

The **proposed system** is an Android-based Blood Donation System designed to overcome the limitations of the existing manual system. The system provides a digital platform that connects blood donors, blood banks, and patients in a simple and efficient way. It helps in managing blood donation activities and locating the required blood group quickly during emergency situations.

In this system, donors can register through the mobile application by providing their personal details, blood group, contact information, and availability status. This information is stored in a centralized database, which makes it easy to access and manage donor records whenever required.

Blood banks can use the system to maintain and update their blood stock details in real time. They can record the available blood groups, track blood units, and manage blood requests efficiently. This helps ensure that accurate information about blood availability is always maintained.

Patients or their relatives can use the application to search for the required blood group and find nearby blood banks or registered donors. They can also send blood requests through the system, which allows donors or blood banks to respond quickly. This feature helps reduce the time required to locate blood during emergencies.

The proposed system also improves communication between donors, blood banks, and patients by providing a centralized and user-friendly platform. It uses mobile technology and digital databases to make the entire process faster, more organized, and more reliable.

Overall, the proposed Blood Donation System helps improve the efficiency of blood management, reduces delays in emergency situations, and encourages voluntary blood donation by making the process easier and more accessible for everyone.

2.3.1 ADVANTAGES OF PROPOSED SYSTEM

1. **Quick Access to Blood Information**
   The system allows patients to quickly search for the required blood group and find available donors or blood banks.
2. **Real-Time Updates**
   Blood banks can update blood stock information instantly, ensuring accurate and up-to-date data.
3. **Centralized Database**
   All donor and blood bank information is stored in a single database, making it easy to manage and access.
4. **Improved Communication**
   The system connects donors, blood banks, and patients on one platform, improving coordination.
5. **Time-Saving**
   Patients do not need to visit multiple hospitals or blood banks to find blood, which saves valuable time.
6. **Easy Donor Registration**
   Donors can easily register and update their availability through the mobile application.
7. **Efficient Data Management**
   Digital records reduce errors and make it easier to maintain donor and blood stock information.
8. **Useful in Emergency Situations**
   The system helps locate blood quickly during emergencies, which can help save lives.
9. **Promotes Blood Donation Awareness**
   The application encourages more people to donate blood and participate in life-saving activities.
10. **User-Friendly System**
    The mobile application is simple and easy to use for donors, blood banks, and patients.

2.4 FEASIBILITY STUDY

A feasibility study is conducted to determine whether the proposed Blood Donation System is practical, beneficial, and possible to implement. It helps evaluate the system from different aspects such as technical, economic, and operational feasibility to ensure that the project can be successfully developed and used.

2.4.1. Technical Feasibility

Technical feasibility refers to the availability of the required technology and resources needed to develop and run the system. The proposed Blood Donation System is an Android-based application that uses commonly available technologies such as mobile devices, internet connectivity, and database management systems. These technologies are widely used and easily accessible, making the development and implementation of the system technically feasible. Developers can design the application using standard programming tools and integrate it with a digital database to store and manage donor and blood bank information.

2.4.2. Economic Feasibility

Economic feasibility evaluates the cost required to develop and maintain the system compared to the benefits it provides. The proposed system does not require expensive hardware or complex infrastructure, as it mainly relies on mobile devices and internet services. The development cost is relatively low compared to the benefits, such as improved blood management, faster communication, and the ability to save lives during emergencies. Therefore, the system is considered economically feasible.

2.4.3. Operational Feasibility

Operational feasibility determines whether the system can be easily used and accepted by its users. The Blood Donation System is designed to be user-friendly so that donors, blood banks, and patients can easily use the application without technical difficulties. The system simplifies the process of registering donors, updating blood stock, and searching for required blood groups. Since the application improves the existing process and makes it more efficient, it is expected to be accepted and widely used by the intended users.

2.5 REQUIREMENT ANALYSIS

Requirement analysis is the process of identifying and understanding the needs and functionalities required to develop the Blood Donation System. It defines what the system should do and the resources needed for its successful implementation. The requirement analysis helps developers design the system according to the needs of donors, blood banks, and patients.

2.5.1. Functional Requirements

Functional requirements describe the main functions and features that the system must perform.

- The system should allow **donors to register** by providing personal details such as name, blood group, contact number, and address.
- Donors should be able to **update their availability status** for blood donation.
- The system should allow **blood banks to register and manage blood stock details**.
- Blood banks should be able to **update available blood units** for different blood groups.
- Patients or their relatives should be able to **search for the required blood group** through the application.
- The system should allow users to **send blood requests** to donors or blood banks.
- The system should **store and manage all data in a centralized database**.

2.5.2. Non-Functional Requirements

Non-functional requirements describe the quality and performance aspects of the system.

- The system should be **user-friendly and easy to operate**.
- The application should provide **fast response time** while searching for blood availability.
- The system should ensure **data security and privacy** for donor information.
- The application should be **reliable and available whenever needed**.
- The system should support **Android mobile devices**.
- The database should be capable of **handling multiple users and storing large amounts of data**.

Overall, the requirement analysis helps ensure that the Blood Donation System meets the needs of users and performs efficiently in managing blood donation activities.

CHAPTER – III

DEVELOPMENT ENVIRONMENT

3.0. DEVELOPMENT ENVIRONMENT

The development environment refers to the hardware and software tools used to design, develop, test, and implement the Blood Donation System. A proper development environment helps developers build the application efficiently and ensures that the system runs smoothly.

3.1. Hardware Requirements

Hardware requirements include the physical devices needed for developing and running the system.

- **Computer System:** A computer with a minimum of Intel i3 processor or higher.
- **RAM:** Minimum 4 GB RAM (8 GB recommended for better performance).
- **Hard Disk:** At least 500 GB storage space.
- **Smartphone Device:** Android mobile phone for testing the application.
- **Internet Connection:** Required for downloading tools, testing, and database connectivity.

3.2. Software Requirements

Software requirements include the programs and tools used to develop the application.

- **Operating System:** Windows 10 / Windows 11
- **Development Tool:** Flutter
- **Programming Language:** dart
- **Database:** Firebase / SQLite

3.3. Development Platform

The Blood Donation System is developed as an **Android mobile application**. Gravity AI is used as the main development environment to design the user interface, write program code, and test the application. The database is used to store donor information, blood bank details, and blood availability records.

The development environment ensures that the application is properly designed, tested, and deployed so that users can access the Blood Donation System easily through their Android devices.

3.4. Conclusion

The **Blood Donation System** is an effective solution designed to improve the process of blood donation and blood distribution using modern mobile technology. The system connects donors, blood banks, and patients on a single platform, making it easier to manage blood availability and donor information.

CHAPTER – IV

DESIGN & DEVELOPMENT

4.0. DESIGN & DEVELOPMENT

The design and development phase focuses on planning the structure of the Blood Donation System and building the application using appropriate technologies. In this phase, the system architecture, user interface, and database structure are designed to ensure that the application works efficiently and is easy to use.

4.1. System Design

System design defines how the different components of the Blood Donation System will interact with each other. The system is designed as an Android mobile application that connects donors, blood banks, and patients through a centralized database. The design ensures that users can easily access the system, search for blood availability, and send requests.

The application is divided into different modules such as **Donor Module, Blood Bank Module, and Patient Module**. Each module performs specific functions such as donor registration, blood stock management, and searching for blood.

4.2. User Interface Design

The user interface (UI) is designed to be simple, clear, and user-friendly. The application includes different screens such as login page, registration page, donor details page, blood search page, and request page. The interface allows users to easily navigate through the application and perform required tasks without difficulty.

4.3. Database Design

A database is used to store all the important information related to donors, blood banks, and blood availability. The database contains details such as donor name, blood group, contact information, and blood stock details. Proper database design ensures that data can be stored, updated, and retrieved quickly.

4.4. Application Development

The development of the Blood Donation System is carried out using **Gravity AI** as the development tool and **Dart** as the programming language. The application is developed by integrating the user interface with the database so that user actions such as registration, searching for blood, and sending requests can be processed efficiently.

4.5. Testing and Implementation

After development, the application is tested to ensure that all features work correctly. Testing helps identify and fix errors or bugs in the system. Once testing is completed successfully, the application can be implemented and used by donors, blood banks, and patients.

Overall, the design and development process ensures that the Blood Donation System is reliable, efficient, and easy to use, helping improve the management of blood donation and availability.

4.6. Chart Module Development

The Chat Module is an important feature of the Blood Donation System that enables direct communication between donors, blood banks, and patients. This module helps users quickly exchange information regarding blood availability, donation requests, and emergency needs. By providing real-time communication, the chat module improves coordination and reduces the time required to arrange blood during critical situations.

The chat module is developed using **Dart programming language**, which is commonly used for building mobile applications. With the help of modern development tools such as **Gravity AI**, the chat functionality is integrated into the application to allow smooth and efficient communication between users.

In this module, users can send and receive text messages through the application interface. When a patient searches for a particular blood group and finds a suitable donor or blood bank, they can directly start a conversation using the chat feature. Similarly, donors and blood bank staff can respond to requests and provide necessary details about blood availability or donation schedules.

The chat system is connected to the application database, where messages are stored and retrieved when required. This ensures that users can view previous conversations and maintain proper communication history. The system also ensures that messages are delivered quickly and reliably through internet connectivity.

The user interface of the chat module is designed to be simple and easy to use. It includes features such as message input, message display, and conversation history. These features help users communicate effectively without any technical difficulty.

Overall, the Chat Module enhances the functionality of the Blood Donation System by providing a quick and convenient communication platform for donors, blood banks, and patients. This feature plays a significant role in improving response time and ensuring that blood donation requests are handled efficiently.

4.7. Search and Filtering Implementation

The **Search and Filtering** feature is an important component of the Blood Donation System that helps users quickly find the required blood group, donors, or blood banks. This feature improves the efficiency of the application by allowing patients and their relatives to easily locate available blood during emergency situations.

The search functionality is implemented using the **Dart programming language**, which allows the application to quickly process user queries and display relevant results. Users can enter the required blood group in the search field, and the system retrieves matching donor or blood bank records from the database.

Filtering is used to narrow down the search results based on specific conditions. For example, users can filter results based on **blood group, location, or donor availability**. This helps users quickly identify suitable donors or nearby blood banks that can provide the required blood.

The application communicates with the database to fetch updated information about donors and blood stock. When a user performs a search, the system checks the stored records and displays only the relevant results that match the search criteria. This ensures that users receive accurate and useful information without unnecessary data.

The user interface for the search and filtering feature is designed to be simple and easy to use. It includes a search bar and filter options that allow users to quickly refine their results. This helps save time and makes the process of finding blood more efficient.

Overall, the Search and Filtering Implementation improves the usability of the Blood Donation System by providing a fast and convenient way for users to locate the required blood group and connect with donors or blood banks quickly.

4.8. Database Integration

Database integration is an important part of the Blood Donation System because it allows the application to store, manage, and retrieve all the required information efficiently. The database acts as the central storage where all details related to donors, blood banks, patients, and blood availability are maintained.

In this project, the application is developed using **Dart**, and the database is integrated with the mobile application to ensure smooth data communication. The database stores important information such as donor name, blood group, contact details, availability status, and blood bank stock details. Whenever a user registers or updates information in the application, the data is automatically stored in the database.

The database also helps in retrieving information quickly when users perform actions such as searching for a specific blood group or sending a blood request. The system sends queries to the database, and the database returns the relevant results, which are then displayed in the application.

Proper database integration ensures that the data is organized, secure, and easily accessible. It also helps in maintaining accurate records of blood donations and blood stock levels. The use of a centralized database makes it possible for donors, blood banks, and patients to access updated information at any time through the application.

Overall, database integration improves the reliability and performance of the Blood Donation System by enabling efficient data storage, quick information retrieval, and smooth communication between different modules of the application.

4.9. Security Implementation Security

Security implementation is an essential part of the Blood Donation System to protect user information and ensure safe access to the application. Since the system stores sensitive data such as donor details, contact information, and blood bank records, proper security measures are required to prevent unauthorized access and misuse of data.

The application includes a **secure login and registration system** to ensure that only authorized users can access the system. Each user must create an account using valid details, and authentication is required before accessing the application features.

Data security is maintained by storing information in a **protected database** where access is restricted to authorized users only. Sensitive information such as personal details and contact numbers is handled carefully to maintain privacy and confidentiality.

The system also includes **input validation** to prevent incorrect or harmful data from being entered into the database. This helps protect the application from common issues such as invalid data entries or malicious inputs.

In addition, the application uses **secure communication through internet connectivity** to ensure that data transferred between the user and the database is protected. Regular updates and proper system maintenance also help in improving the overall security of the application.

Overall, the security implementation ensures that the Blood Donation System is safe, reliable, and trustworthy for donors, blood banks, and patients. It protects user data, prevents unauthorized access, and ensures that the system operates securely.

4.10. Performance Optimization To

Performance optimization is an important aspect of the Blood Donation System to ensure that the application runs smoothly, quickly, and efficiently for all users. A well-optimized system improves user experience by reducing delays, improving response time, and ensuring reliable performance even when multiple users access the system.

In this project, the application is developed using **Dart**, which helps in building fast and responsive mobile applications. Proper coding practices are followed to ensure that the application performs efficiently and uses system resources effectively.

One of the key optimization techniques used in the system is **efficient database management**. Data is stored in a structured format so that it can be retrieved quickly when users perform actions such as searching for donors or checking blood availability. Optimized database queries help reduce the time required to fetch information from the database.

The application also uses **search and filtering optimization**, which ensures that only relevant data is displayed to the user. This reduces unnecessary processing and improves the speed of search results.

Another important optimization method is **reducing unnecessary data loading**. The system loads only the required information when needed instead of loading large amounts of data at once. This improves application performance and reduces memory usage.

In addition, proper testing and debugging are carried out during development to identify performance issues and fix them. Regular updates and improvements help maintain the efficiency of the application.

Overall, performance optimization ensures that the Blood Donation System provides fast response times, smooth operation, and a reliable experience for donors, blood banks, and patients using the application.

4.11 Conclusion

The Blood Donation System is a modern mobile-based solution designed to simplify and improve the process of blood donation and blood distribution. The system connects donors, blood banks, and patients through a single digital platform, making it easier to manage blood availability and donation activities.

By using technologies such as **Dart** and **Gravity AI**, the application provides a fast, efficient, and user-friendly environment for managing blood donation services. Features such as donor registration, blood search, chat communication, and real-time database integration help users quickly access the information they need.

The system reduces the problems faced in traditional manual methods, such as delays in finding blood, lack of communication, and difficulty in managing donor records. With the help of search and filtering features, patients can easily find the required blood group and contact donors or blood banks quickly, especially during emergency situations.

In addition, the application promotes awareness about voluntary blood donation and encourages more people to participate in saving lives. Proper security measures and performance optimization also ensure that the system operates safely

CHAPTER – V

CODE SECTION

5.0. Table of Contents

Root

main.dart

Core

core/router/app_router.dart core/theme/app_theme.dart

Features

features/auth/data/auth_service.dart features/auth/presentation/screens/login_screen.dart
features/auth/presentation/screens/register_screen.dart
features/auth/presentation/screens/role_selection_screen.dart
features/dashboard/presentation/admin/screens/admin_dashboard_screen.dart
features/dashboard/presentation/donor/screens/donor_dashboard_screen.dart
features/dashboard/presentation/receiver/screens/receiver_dashboard_screen.dart
features/blood_requests/presentation/screens/blood_requests_screen.dart
features/blood_requests/presentation/screens/blood_request_detail_screen.dart
features/blood_requests/presentation/screens/post_blood_request_screen.dart
features/profile/presentation/screens/profile_screen.dart
features/notifications/presentation/screens/notifications_screen.dart

Rbac

rbac/rbac_service.dart

Shared

shared/services/user_model.dart

5.0.1. CORE

app_router.dart

Path: core/router/app_router.dart | Lines (stripped): 160

```
import 'package:flutter/material.dart';

import 'package:flutter_riverpod/flutter_riverpod.dart'; import 'package:go_router/go_router.dart'; import
'../../rbac/models/app_role.dart';

import '../../features/auth/presentation/providers/local_auth_provider.dart'; import
'../../features/auth/presentation/screens/login_screen.dart'; import
'../../features/auth/presentation/screens/register_screen.dart'; import
'../../features/auth/presentation/screens/role_selection_screen.dart';

import '../../features/verification/presentation/screens/verification_pending_screen.dart'; import
'../../features/verification/presentation/screens/verification_upload_screen.dart'; import
'../../features/verification/presentation/screens/verification_review_screen.dart'; import
'../../features/dashboard/presentation/admin/screens/admin_dashboard_screen.dart'; import
'../../features/dashboard/presentation/verifier/screens/verifier_dashboard_screen.dart'; import
'../../features/dashboard/presentation/donor/screens/donor_dashboard_screen.dart'; import
'../../features/dashboard/presentation/receiver/screens/receiver_dashboard_screen.dart'; import
'../../features/blood_requests/presentation/screens/blood_requests_screen.dart'; import
'../../features/blood_requests/presentation/screens/post_blood_request_screen.dart'; import
'../../features/blood_requests/presentation/screens/my_requests_screen.dart'; import
'../../features/blood_requests/presentation/screens/blood_request_detail_screen.dart'; import
'../../features/notifications/presentation/screens/notifications_screen.dart'; import
'../../features/profile/presentation/screens/profile_screen.dart'; import
'../../features/profile/presentation/screens/edit_profile_screen.dart'; import '../../shared/widgets/main_scaffold.dart';
import 'app_routes.dart';

final routerProvider = Provider<GoRouter>((ref) { final authState = ref.watch(localAuthStateProvider); return
GoRouter(

initialLocation: AppRoutes.splash, redirect: (context, state) {
```

```
final isLoggedIn = authState.valueOrNull != null;

final isOnAuth = state.matchedLocation == AppRoutes.login || state.matchedLocation == AppRoutes.register || state.matchedLocation == AppRoutes.roleSelection || state.matchedLocation == AppRoutes.splash; if (!isLoggedIn && !isOnAuth) { return AppRoutes.login; }

if (isLoggedIn && isOnAuth) {

final role = authState.valueOrNull?.role;

final isVerified = authState.valueOrNull?.isVerified ?? false;

if (!isVerified && role != AppRole.superAdmin && role != AppRole.admin && role != AppRole.verifier) { return AppRoutes.verificationPending; }

return _getDashboardRoute(role); }

return null; }, routes: [ GoRoute(

path: AppRoutes.splash,

builder: (context, state) => const SplashScreen(), ), GoRoute(

path: AppRoutes.login,

builder: (context, state) => const LoginScreen(), ), GoRoute(

path: AppRoutes.register,

builder: (context, state) => const RegisterScreen(),

),
```

```
GoRoute(
path: AppRoutes.roleSelection,
builder: (context, state) => const RoleSelectionScreen(), ), GoRoute(
path: AppRoutes.verificationPending,
builder: (context, state) => const VerificationPendingScreen(), ), GoRoute(
path: AppRoutes.verificationUpload,
builder: (context, state) => const VerificationUploadScreen(), ), GoRoute(
path: '/verification-review/:uid', builder: (context, state) {
final uid = state.pathParameters['uid'] ?? ''; return VerificationReviewScreen(uid: uid); }, ), ShellRoute(
builder: (context, state, child) => MainScaffold(child: child), routes: [ GoRoute(
path: AppRoutes.adminDashboard,
builder: (context, state) => const AdminDashboardScreen(), ), GoRoute(
path: AppRoutes.verifierDashboard,
builder: (context, state) => const VerifierDashboardScreen(), ), GoRoute(
path: AppRoutes.donorDashboard,
builder: (context, state) => const DonorDashboardScreen(), ), GoRoute(
path: AppRoutes.receiverDashboard,
builder: (context, state) => const ReceiverDashboardScreen(), ), GoRoute(
path: AppRoutes.bloodRequests,
builder: (context, state) => const BloodRequestsScreen(), ), GoRoute(
path: '/blood-request/:id', builder: (context, state) {
final id = state.pathParameters['id'] ?? ''; return BloodRequestDetailScreen(requestId: id); }, ), GoRoute(
path: AppRoutes.postBloodRequest,
builder: (context, state) => const PostBloodRequestScreen(), ), GoRoute(
path: AppRoutes.myRequests,
builder: (context, state) => const MyRequestsScreen(), ), GoRoute(
path: AppRoutes.notifications,
builder: (context, state) => const NotificationsScreen(),
```

```
), GoRoute(

path: AppRoutes.profile,

builder: (context, state) => const ProfileScreen(), ), GoRoute(

path: AppRoutes.editProfile,

builder: (context, state) => const EditProfileScreen(), ), ], ), ], );

}); String _getDashboardRoute(AppRole? role) { switch (role) { case AppRole.superAdmin: case AppRole.admin:

return AppRoutes.adminDashboard; case AppRole.verifier:

return AppRoutes.verifierDashboard; case AppRole.donor:

return AppRoutes.donorDashboard; case AppRole.receiver:

return AppRoutes.receiverDashboard; default:

return AppRoutes.login; } }

class SplashScreen extends StatelessWidget { const SplashScreen({super.key}); @override Widget build(BuildContext
context) { return const Scaffold( body: Center(

child: CircularProgressIndicator(), ),

); } }
```

5.1 app_theme.dart

Path: core/theme/app_theme.dart | Lines (stripped): 215

import 'package:flutter/material.dart'; import '../constants/app_colors.dart'; import '../constants/app_radius.dart'; import '../constants/app_typography.dart'; class AppTheme { AppTheme._(); static ThemeData get light {

return ThemeData(useMaterial3: true, brightness: Brightness.light, primaryColor: AppColors.primary,

scaffoldBackgroundColor: AppColors.surface, colorScheme: const ColorScheme.light(primary: AppColors.primary, secondary: AppColors.secondary, surface: AppColors.surface, error: AppColors.error, onPrimary: Colors.white, onSecondary: Colors.white, onSurface: AppColors.onSurface, onError: Colors.white,),

appBarTheme: const AppBarTheme(backgroundColor: AppColors.surface, foregroundColor: AppColors.onSurface, elevation: 0, centerTitle: true,

titleTextStyle: AppTypography.headline5,),

cardTheme: CardThemeData(color: AppColors.cardBackground, elevation: 2,

shape: RoundedRectangleBorder(

borderRadius: BorderRadius.circular(AppRadius.md),),),

elevatedButtonTheme: ElevatedButtonThemeData(style: ElevatedButton.styleFrom(backgroundColor: AppColors.primary, foregroundColor: Colors.white,

minimumSize: const Size(double.infinity, 48), shape: RoundedRectangleBorder(

borderRadius: BorderRadius.circular(AppRadius.sm),),

textStyle: AppTypography.button,),),

outlinedButtonTheme: OutlinedButtonThemeData(style: OutlinedButton.styleFrom(foregroundColor: AppColors.primary, minimumSize: const Size(double.infinity, 48), side: const BorderSide(color: AppColors.primary), shape: RoundedRectangleBorder(

borderRadius: BorderRadius.circular(AppRadius.sm),),

textStyle: AppTypography.button,),),

textButtonTheme: TextButtonThemeData(style: TextButton.styleFrom(

foregroundColor: AppColors.primary, textStyle: AppTypography.button,),),

inputDecorationTheme: InputDecorationTheme(filled: true,

fillColor: Colors.white,

contentPadding: const EdgeInsets.symmetric(horizontal: 16, vertical: 16), border: OutlineInputBorder(

borderRadius: BorderRadius.circular(AppRadius.sm), borderSide: const BorderSide(color: AppColors.divider),),,

```
return ThemeData( useMaterial3: true, brightness: Brightness.light, primaryColor: AppColors.primary,

scaffoldBackgroundColor: AppColors.surface, colorScheme: const ColorScheme.light( primary: AppColors.primary, secondary: AppColors.secondary, surface: AppColors.surface, error: AppColors.error, onPrimary: Colors.white, onSecondary: Colors.white, onSurface: AppColors.onSurface, onError: Colors.white, ),

appBarTheme: const AppBarTheme( backgroundColor: AppColors.surface, foregroundColor: AppColors.onSurface, elevation: 0, centerTitle: true,

titleTextStyle: AppTypography.headline5, ),

cardTheme: CardThemeData( color: AppColors.cardBackground, elevation: 2,

shape: RoundedRectangleBorder(

borderRadius: BorderRadius.circular(AppRadius.md), ), ),

elevatedButtonTheme: ElevatedButtonThemeData( style: ElevatedButton.styleFrom( backgroundColor: AppColors.primary, foregroundColor: Colors.white,

minimumSize: const Size(double.infinity, 48), shape: RoundedRectangleBorder(

borderRadius: BorderRadius.circular(AppRadius.sm), ),

textStyle: AppTypography.button, ), ),

outlinedButtonTheme: OutlinedButtonThemeData( style: OutlinedButton.styleFrom( foregroundColor: AppColors.primary, minimumSize: const Size(double.infinity, 48), side: const BorderSide(color: AppColors.primary), shape: RoundedRectangleBorder(

borderRadius: BorderRadius.circular(AppRadius.sm), ),

textStyle: AppTypography.button, ), ),

textButtonTheme: TextButtonThemeData( style: TextButton.styleFrom(

foregroundColor: AppColors.primary, textStyle: AppTypography.button, ), ),

inputDecorationTheme: InputDecorationTheme( filled: true,

fillColor: Colors.white,

contentPadding: const EdgeInsets.symmetric(horizontal: 16, vertical: 16), border: OutlineInputBorder(

borderRadius: BorderRadius.circular(AppRadius.sm), borderSide: const BorderSide(color: AppColors.divider), ),

enabledBorder: OutlineInputBorder(

borderRadius: BorderRadius.circular(AppRadius.sm), borderSide: const BorderSide(color: AppColors.divider), ),

focusedBorder: OutlineInputBorder(

borderRadius: BorderRadius.circular(AppRadius.sm),
```

```
borderSide: const BorderSide(color: AppColors.primary, width: 2), ),

errorBorder: OutlineInputBorder(

borderRadius: BorderRadius.circular(AppRadius.sm), borderSide: const BorderSide(color: AppColors.error), ),

focusedErrorBorder: OutlineInputBorder(

borderRadius: BorderRadius.circular(AppRadius.sm),

borderSide: const BorderSide(color: AppColors.error, width: 2), ), ),

bottomNavigationBarTheme: const BottomNavigationBarThemeData( backgroundColor: Colors.white,
selectedItemColor: AppColors.primary, unselectedItemColor: Colors.grey, type: BottomNavigationBarType.fixed,
elevation: 8, ),

dividerTheme: const DividerThemeData( color: AppColors.divider, thickness: 1, ),

snackBarTheme: SnackBarThemeData( backgroundColor: AppColors.onSurface,

contentTextStyle: AppTypography.bodyMedium.copyWith(color: Colors.white), behavior: SnackBarBehavior.floating,
shape: RoundedRectangleBorder(

borderRadius: BorderRadius.circular(AppRadius.sm), ), ),

); }

static ThemeData get dark { return ThemeData( useMaterial3: true, brightness: Brightness.dark, primaryColor:
AppColors.primary,

scaffoldBackgroundColor: AppColors.surfaceDark, colorScheme: const ColorScheme.dark( primary:
AppColors.primary, secondary: AppColors.secondary, surface: AppColors.surfaceDark,
```

```
focusedBorder: OutlineInputBorder(

borderRadius: BorderRadius.circular(AppRadius.sm),

borderSide: const BorderSide(color: AppColors.primaryLight, width: 2), ),

errorBorder: OutlineInputBorder(

borderRadius: BorderRadius.circular(AppRadius.sm), borderSide: const BorderSide(color: AppColors.error), ),

focusedErrorBorder: OutlineInputBorder(

borderRadius: BorderRadius.circular(AppRadius.sm),

borderSide: const BorderSide(color: AppColors.error, width: 2), ), ),

bottomNavigationBarTheme: const BottomNavigationBarThemeData( backgroundColor:
AppColors.cardBackgroundDark, selectedItemColor: AppColors.primaryLight, unselectedItemColor: Colors.grey, type:
BottomNavigationBarType.fixed, elevation: 8, ),

dividerTheme: const DividerThemeData( color: AppColors.dividerDark, thickness: 1, ),

snackBarTheme: SnackBarThemeData( backgroundColor: AppColors.onSurfaceDark,

contentTextStyle: AppTypography.bodyMedium.copyWith(color: Colors.black), behavior: SnackBarBehavior.floating,
shape: RoundedRectangleBorder(

borderRadius: BorderRadius.circular(AppRadius.sm), ), ),

); } }
```

5.2. FEATURES

5.2.1 auth_service.dart

Path: features/auth/data/auth_service.dart | Lines (stripped): 28

```
import 'package:fpdart/fpdart.dart';

import 'package:flutter_secure_storage/flutter_secure_storage.dart'; import '../../../core/errors/failures.dart'; import '../../../rbac/models/app_role.dart'; import '../../../shared/services/user_model.dart'; class AuthService {

final FlutterSecureStorage _secureStorage = const FlutterSecureStorage(); static const String _userIdKey = 'user_id'; Future<Either<Failure, String>> signInWithPhone(String phone) async { return const Right('local_verification'); } Future<Either<Failure, User>> verifyOTP(String verificationId, String otp) async { return const Left(AuthFailure('Firebase auth removed - use local auth')); } Future<Either<Failure, User>> registerWithPhone(String phone, AppRole role) async { return const Left(AuthFailure('Firebase auth removed - use local auth')); } Future<Either<Failure, User?>> getCurrentUser() async { return const Right(null); } Future<void> signOut() async {

await _secureStorage.delete(key: _userIdKey); await _secureStorage.delete(key: 'verification_id'); } Future<Either<Failure, User>> updateUserRole(AppRole role) async { return const Left(AuthFailure('Firebase auth removed - use local auth')); } }
```

5.2.2 login_screen.dart

Path: features/auth/presentation/screens/login_screen.dart | Lines (stripped): 137

```
import 'package:flutter/material.dart';

import 'package:flutter_riverpod/flutter_riverpod.dart'; import 'package:go_router/go_router.dart'; import
'../../../../core/constants/app_colors.dart'; import '../../../../core/constants/app_spacing.dart'; import
'../../../../core/router/app_routes.dart'; import '../providers/local_auth_provider.dart'; class LoginScreen extends
ConsumerStatefulWidget { const LoginScreen({super.key}); @override ConsumerState<LoginScreen> createState() =>
_LoginScreenState(); }

class _LoginScreenState extends ConsumerState<LoginScreen> with SingleTickerProviderStateMixin { final
_phoneController = TextEditingController(); final _formKey = GlobalKey<FormState>(); bool _isLoading = false;

late AnimationController _shakeController; late Animation<double> _shakeAnimation; @override void initState() {
super.initState(); _shakeController = AnimationController( duration: const Duration(milliseconds: 500), vsync: this, );

_shakeAnimation = Tween<double>(begin: 0, end: 10).animate( CurvedAnimation(parent: _shakeController, curve:
Curves.elasticIn),

); } @override void dispose() { _phoneController.dispose(); _shakeController.dispose(); super.dispose(); }

Future<void> _login() async {
```

```
if (!_formKey.currentState!.validate()) return; setState(() => _isLoading = true);
await ref.read(localAuthStateProvider.notifier).login(_phoneController.text); setState(() => _isLoading = false);
final authState = ref.read(localAuthStateProvider); authState.when( data: (user) { if (user != null) {
if (user.role.name == 'donor' || user.role.name == 'receiver') { context.go(AppRoutes.roleSelection); } else {
context.go(AppRoutes.verificationPending); }
} else { _shakeController.forward().then((_) => _shakeController.reset()); } },
loading: () {}, error: (_, _) { _shakeController.forward().then((_) => _shakeController.reset()); },
); }
@override
return 'Please enter your phone number'; }
if (value.length < 10) {
return 'Please enter a valid phone number'; }
return null; }, ),
const SizedBox(height: AppSpacing.lg), ElevatedButton(
onPressed: _isLoading ? null : _login, child: _isLoading ? const SizedBox(height: 20, width: 20, child: CircularProgressIndicator(strokeWidth:
2)) : const Text('Login'), ),
const SizedBox(height: AppSpacing.md), TextButton(
onPressed: () => context.push(AppRoutes.register), child: const Text("Don't have an account? Register"), ),
const Spacer(), ], ), ),
), ), ),
); } }
```

5.2.3. register_screen.dart

Path: features/auth/presentation/screens/register_screen.dart | Lines (stripped): 121

```
setState(() => _isLoading = false); if (mounted) {
final authState = ref.read(localAuthStateProvider); authState.when( data: (user) { if (user != null) {
context.go(AppRoutes.roleSelection); } },
loading: () {}, error: (_, _) {}, ); } }
@override Widget build(BuildContext context) { return Scaffold(
appBar: AppBar(title: const Text('Register')), body: SafeArea( child: Padding(
padding: const EdgeInsets.all(AppSpacing.lg), child: Form( key: _formKey, child: Column(
```

```
mainAxisAlignment: MainAxisAlignment.center, crossAxisAlignment: CrossAxisAlignment.stretch, children: [ if
(!_otpSent) ...[ TextFormField(

controller: _phoneController, keyboardType: TextInputType.phone, decoration: const InputDecoration( labelText: 'Phone
Number', hintText: '+91 9876543210', prefixIcon: Icon(Icons.phone), ),

validator: (value) {

if (value == null || value.isEmpty) return 'Required'; if (value.length < 10) return 'Invalid number'; return null; }, ),

const SizedBox(height: AppSpacing.lg), ElevatedButton(

onPressed: _isLoading ? null : _sendOTP, child: _isLoading ? const SizedBox(height: 20, width: 20, child:
CircularProgressIndicator(strokeWidth:

2)) : const Text('Send OTP'), ),

] else ...[ Card( child: Padding(

padding: const EdgeInsets.all(AppSpacing.md), child: Column( children: [

const Icon(Icons.check_circle, color: AppColors.success, size: 48), const SizedBox(height: AppSpacing.sm), Text('OTP
sent to ${_phoneController.text}', style: Theme.of(context).textTheme.body Large),

const SizedBox(height: AppSpacing.sm),

const Text('Use any 6-digit code to verify', style: TextStyle(color: Colors.grey)), ], ),

), ),

const SizedBox(height: AppSpacing.lg), ElevatedButton(

onPressed: _isLoading ? null : _verifyAndRegister, child: _isLoading ? const SizedBox(height: 20, width: 20, child:
CircularProgressIndicator(strokeWidth:

2)) : const Text('Verify & Register'), ),

const SizedBox(height: AppSpacing.md), TextButton(

onPressed: () => setState(() => _otpSent = false), child: const Text('Change Phone Number'), ), ], ], ), ), ),
),

); }

}
```

5.2.4. role_selection_screen.dart

Path: features/auth/presentation/screens/role_selection_screen.dart | Lines (stripped): 116

```
import 'package:flutter/material.dart';

import 'package:flutter_riverpod/flutter_riverpod.dart'; import 'package:go_router/go_router.dart'; import
'../../../../core/constants/app_colors.dart'; import '../../../../core/constants/app_spacing.dart'; import
'../../../../core/constants/app_radius.dart'; import '../../../../core/router/app_routes.dart'; import
'../../../../rbac/models/app_role.dart'; import '../providers/local_auth_provider.dart';

class RoleSelectionScreen extends ConsumerStatefulWidget { const RoleSelectionScreen({super.key}); @override
ConsumerState<RoleSelectionScreen> createState() => _RoleSelectionScreenState(); }

class _RoleSelectionScreenState extends ConsumerState<RoleSelectionScreen> { AppRole? _selectedRole; bool
_isLoading = false; Future<void> _continue() async { if (_selectedRole == null) return; setState(() => _isLoading =
true);

await ref.read(localAuthStateProvider.notifier).updateRole(_selectedRole!); setState(() => _isLoading = false); if
(mounted) {

context.go(AppRoutes.verificationPending); } }

@override Widget build(BuildContext context) { return Scaffold(

appBar: AppBar(title: const Text('Select Your Role')), body: SafeArea( child: Padding(

padding: const EdgeInsets.all(AppSpacing.lg), child: Column(

crossAxisAlignment: CrossAxisAlignment.stretch, children: [

const SizedBox(height: AppSpacing.lg), Text('I want to', style: Theme.of(context).textTheme.headlineSmall, textAlign:
TextAlign.center) ,

const SizedBox(height: AppSpacing.xl), _RoleCard(

icon: Icons.bloodtype, title: 'Donate Blood',

description: 'Help save lives by donating blood', isSelected: _selectedRole == AppRole.donor,

onTap: () => setState(() => _selectedRole = AppRole.donor), ),

const SizedBox(height: AppSpacing.md), _RoleCard(

icon: Icons.person_search,
```

5.2.5. admin_dashboard_screen.dart

Path: features/dashboard/presentation/admin/screens/admin_dashboard_screen.dart | Lines (stripped): 167

```
import 'package:flutter/material.dart';

import 'package:flutter_riverpod/flutter_riverpod.dart'; import '../../../../../core/constants/app_colors.dart'; import '../../../../../core/constants/app_spacing.dart'; import '../../../../../shared/services/local_admin_service.dart'; import '../../../../../shared/services/user_model.dart';

final adminUsersProvider = FutureProvider<List<User>>((ref) async { final service = ref.read(localAdminServiceProvider); final result = await service.getAllUsers(); return result.fold((l) => [], (r) => r); });

final adminStatsProvider = FutureProvider<Map<String, dynamic>>((ref) async { final service = ref.read(localAdminServiceProvider); final result = await service.getStats(); return result.fold((l) => {}, (r) => r); });

class AdminDashboardScreen extends ConsumerWidget { const AdminDashboardScreen({super.key}); @override Widget build(BuildContext context, WidgetRef ref) { final statsAsync = ref.watch(adminStatsProvider); final usersAsync = ref.watch(adminUsersProvider); return Scaffold(

appBar: AppBar(title: const Text('Admin Dashboard')), body: SafeArea(

child: SingleChildScrollView(

padding: const EdgeInsets.all(AppSpacing.lg), child: Column(

crossAxisAlignment: CrossAxisAlignment.stretch, children: [ statsAsync.when(

data: (stats) => _StatsGrid(stats: stats),

loading: () => const Center(child: CircularProgressIndicator()), error: (e, _) => Text('Error: $e'), ),

const SizedBox(height: AppSpacing.lg), Text('User Management', style: Theme.of(context).textTheme.titleLarge), const SizedBox(height: AppSpacing.md), usersAsync.when(

data: (users) => ListView.builder( shrinkWrap: true,

physics: const NeverScrollableScrollPhysics(), itemCount: users.length,

itemBuilder: (context, index) => _UserCard(user: users[index]), ),

loading: () => const Center(child: CircularProgressIndicator()), error: (e, _) => Text('Error: $e'), ), ], ), ), ),

); } }

class _StatsGrid extends StatelessWidget { final Map<String, dynamic> stats; const _StatsGrid({required this.stats}); @override

Widget build(BuildContext context) {
```

```
return GridView.count( crossAxisCount: 2, shrinkWrap: true,

physics: const NeverScrollableScrollPhysics(), mainAxisSpacing: AppSpacing.md, crossAxisSpacing: AppSpacing.md,
childAspectRatio: 1.5, children: [

_StatCard(title: 'Total Users', value: '${stats['totalUsers'] ?? 0}', icon: Icons.people, color: AppCo
```

```
lors.secondary), _StatCard(title: 'Pending', value: '${stats['pendingVerifications'] ?? 0}', icon: Icons.pending, color

: AppColors.warning), _StatCard(title: 'Verified', value: '${stats['verifiedUsers'] ?? 0}', icon: Icons.verified, color: App

Colors.success), _StatCard(title: 'Donors', value: '${stats['donors'] ?? 0}', icon: Icons.bloodtype, color: AppColors.p
rimary), ],

); } }

class _StatCard extends StatelessWidget { final String title; final String value; final IconData icon; final Color color;

const _StatCard({required this.title, required this.value, required this.icon, required this.color}); @override Widget
build(BuildContext context) { return Card( child: Padding(

padding: const EdgeInsets.all(AppSpacing.md), child: Column(

mainAxisAlignment: MainAxisAlignment.center, children: [ Icon(icon, color: color, size: 28), const SizedBox(height:
AppSpacing.xs), Text(value, style: Theme.of(context).textTheme.headlineSmall?.copyWith(fontWeight: FontWeight.bold

, color: color)), Text(title, style: Theme.of(context).textTheme.bodySmall, textAlign: TextAlign.center), ], ), ),

); } }

class _UserCard extends ConsumerWidget { final User user;

const _UserCard({required this.user}); @override Widget build(BuildContext context, WidgetRef ref) { return Card(

margin: const EdgeInsets.only(bottom: AppSpacing.sm), child: ListTile( leading: CircleAvatar(

backgroundColor: _getStatusColor().withValues(alpha: 0.1), child: Icon(Icons.person, color: _getStatusColor()), ), title:
Row( children: [

Text(user.name ?? user.phone),
```

```
if (user.isBanned) ...[ const SizedBox(width: 8), Container(

padding: const EdgeInsets.symmetric(horizontal: 6, vertical: 2),

decoration: BoxDecoration(color: AppColors.error, borderRadius: BorderRadius.circular(4)), child: const
Text('BANNED', style: TextStyle(color: Colors.white, fontSize: 10)), ), ], ], ),

subtitle: Text('${user.role.displayName} • ${user.verificationStatus.displayName}'), trailing: PopupMenuButton(
itemBuilder: (context) => [ PopupMenuItem( value: 'ban', child: Row( children: [ Icon(user.isBanned ? Icons.lock_open :
Icons.block, size: 20), const SizedBox(width: 8),

Text(user.isBanned ? 'Unban' : 'Ban'),
```

```
], ), ), ],

onSelected: (value) async {

final service = ref.read(localAdminServiceProvider); if (value == 'ban') { if (user.isBanned) {

await service.unbanUser(user.uid); } else {

await service.banUser(user.uid); }

ref.invalidate(adminUsersProvider); ref.invalidate(adminStatsProvider); }

}, ), ),

);

} Color _getStatusColor() { switch (user.verificationStatus) { case VerificationStatus.verified:

return AppColors.success; case VerificationStatus.pending: case VerificationStatus.inReview:

return AppColors.warning; case VerificationStatus.rejected:

return AppColors.error; } } }
```

5.2.6. donor_dashboard_screen.dart

Path: features/dashboard/presentation/donor/screens/donor_dashboard_screen.dart | Lines (stripped): 150

```
import 'package:flutter/material.dart';

import 'package:flutter_riverpod/flutter_riverpod.dart'; import '../../../../../core/constants/app_colors.dart'; import '../../../../../core/constants/app_spacing.dart'; import '../../../../../core/router/app_routes.dart';

import '../../../../auth/presentation/providers/local_auth_provider.dart'; import 'package:go_router/go_router.dart'; class DonorDashboardScreen extends ConsumerWidget { const DonorDashboardScreen({super.key}); @override Widget build(BuildContext context, WidgetRef ref) { final authState = ref.watch(localAuthStateProvider); final user = authState.valueOrNull; return Scaffold( appBar: AppBar(

title: const Text('Donor Dashboard'), ),

body: SafeArea(

child: SingleChildScrollView(

padding: const EdgeInsets.all(AppSpacing.lg), child: Column(

crossAxisAlignment: CrossAxisAlignment.stretch, children: [ _AvailabilityToggle(user: user), const SizedBox(height: AppSpacing.lg), _QuickActions(context: context), ], ), ), ),

); } }

class _AvailabilityToggle extends StatelessWidget { final dynamic user;
```

```
const _AvailabilityToggle({this.user}); @override Widget build(BuildContext context) {
final isAvailable = user?.isAvailable ?? false; return Card( child: Padding(
padding: const EdgeInsets.all(AppSpacing.lg), child: Column( children: [ Container(
padding: const EdgeInsets.all(AppSpacing.md), decoration: BoxDecoration(
color: isAvailable ? AppColors.success.withValues(alpha: 0.1) : AppColors.warning.withValues(a lpha: 0.1),
shape: BoxShape.circle, ), child: Icon(
isAvailable ? Icons.check_circle : Icons.cancel,
color: isAvailable ? AppColors.success : AppColors.warning, size: 48, ), ),
const SizedBox(height: AppSpacing.md), Text(
isAvailable ? 'You are Available' : 'You are Unavailable',
```

```
style: Theme.of(context).textTheme.titleLarge?.copyWith( fontWeight: FontWeight.bold, ), ),

const SizedBox(height: AppSpacing.sm), Text( isAvailable ? 'Blood banks and receivers can see you as available for donation' : 'Toggle to let others know you are available to donate',

style: Theme.of(context).textTheme.bodySmall?.copyWith(color: Colors.grey), textAlign: TextAlign.center, ),

const SizedBox(height: AppSpacing.md), Switch(

value: isAvailable, onChanged: (value) {},

activeThumbColor: AppColors.success, ),

], ), ),

); } }

class _QuickActions extends StatelessWidget { final BuildContext context;

const _QuickActions({required this.context}); @override Widget build(BuildContext context) { return Column(

crossAxisAlignment: CrossAxisAlignment.start, children: [ Text( 'Quick Actions',

style: Theme.of(context).textTheme.titleMedium?.copyWith( fontWeight: FontWeight.bold, ), ),

const SizedBox(height: AppSpacing.md), _ActionCard( icon: Icons.bloodtype, title: 'Browse Blood Requests', subtitle: 'Find people who need blood', color: AppColors.primary,

onTap: () => context.go(AppRoutes.bloodRequests),

),
```

```
const SizedBox(height: AppSpacing.md), _ActionCard( icon: Icons.history, title: 'My Donations',

subtitle: 'View your donation history', color: AppColors.secondary, onTap: () {}, ), ],

); } }

class _ActionCard extends StatelessWidget { final IconData icon;
```

```
final String title; final String subtitle; final Color color; final VoidCallback onTap; const _ActionCard({ required this.icon, required this.title, required this.subtitle, required this.color, required this.onTap, });

@override Widget build(BuildContext context) { return Card( child: ListTile( onTap: onTap, leading: Container(

padding: const EdgeInsets.all(AppSpacing.sm), decoration: BoxDecoration( color: color.withValues(alpha: 0.1), borderRadius: BorderRadius.circular(8), ),

child: Icon(icon, color: color), ),

title: Text(title), subtitle: Text(subtitle),

trailing: const Icon(Icons.chevron_right), ),

); } }
```

5.2.7. receiver_dashboard_screen.dart

Path: features/dashboard/presentation/receiver/screens/receiver_dashboard_screen.dart | Lines (stripped): 82

```
import 'package:flutter/material.dart';

import 'package:flutter_riverpod/flutter_riverpod.dart'; import '../../../../../core/constants/app_colors.dart'; import '../../../../../core/constants/app_spacing.dart'; import '../../../../../core/router/app_routes.dart'; import 'package:go_router/go_router.dart';

class ReceiverDashboardScreen extends ConsumerWidget { const ReceiverDashboardScreen({super.key}); @override Widget build(BuildContext context, WidgetRef ref) { return Scaffold( appBar: AppBar(

title: const Text('Receiver Dashboard'), ),

body: SafeArea(

child: SingleChildScrollView(

padding: const EdgeInsets.all(AppSpacing.lg), child: Column(

crossAxisAlignment: CrossAxisAlignment.stretch, children: [ Text(

'Quick Actions',
```

```
style: Theme.of(context).textTheme.titleMedium?.copyWith( fontWeight: FontWeight.bold, ), ),

const SizedBox(height: AppSpacing.md), _ActionCard(

icon: Icons.add_circle, title: 'Post Blood Request',

subtitle: 'Request blood for yourself or a patient', color: AppColors.primary,

onTap: () => context.go(AppRoutes.postBloodRequest), ),

const SizedBox(height: AppSpacing.md), _ActionCard( icon: Icons.list_alt, title: 'My Requests',

subtitle: 'View status of your blood requests', color: AppColors.secondary,

onTap: () => context.go(AppRoutes.myRequests), ), ], ), ), ),

); } }

class _ActionCard extends StatelessWidget { final IconData icon; final String title; final String subtitle; final Color color;
final VoidCallback onTap; const _ActionCard({ required this.icon, required this.title, required this.subtitle, required
this.color,

required this.onTap, });

@override Widget build(BuildContext context) { return Card( child: ListTile( onTap: onTap, leading: Container(

padding: const EdgeInsets.all(AppSpacing.sm), decoration: BoxDecoration( color: color.withValues(alpha: 0.1),
borderRadius: BorderRadius.circular(8), ),

child: Icon(icon, color: color), ),

title: Text(title), subtitle: Text(subtitle),

trailing: const Icon(Icons.chevron_right), ),

); } }
```

5.2.8. blood_requests_screen.dart

Path: features/blood_requests/presentation/screens/blood_requests_screen.dart | Lines (stripped): 253

import 'package:flutter/material.dart'; import 'package:flutter_riverpod/flutter_riverpod.dart'; import '../../../../core/constants/app_colors.dart'; import '../../../../core/constants/app_spacing.dart'; import '../../../../core/constants/app_radius.dart'; import '../../../../shared/services/blood_request_model.dart'; import 'package:shimmer/shimmer.dart';

final bloodRequestsProvider = FutureProvider<List<BloodRequest>>((ref) async { return []; });

class BloodRequestsScreen extends ConsumerWidget { const BloodRequestsScreen({super.key}); @override Widget build(BuildContext context, WidgetRef ref) { final requestsAsync = ref.watch(bloodRequestsProvider); return Scaffold(appBar: AppBar(

title: const Text('Blood Requests'),),

body: SafeArea(child: Column(children: [_BloodTypeFilter(), Expanded(

child: requestsAsync.when(data: (requests) => requests.isEmpty ? _EmptyState() : ListView.builder(

padding: const EdgeInsets.all(AppSpacing.md), itemCount: requests.length, itemBuilder: (context, index) {

return _BloodRequestCard(request: requests[index]); },),

loading: () => _LoadingState(),

error: (error, stack) => _ErrorState(message: error.toString()),),),],),),

); } }

class _BloodTypeFilter extends StatefulWidget { @override State<_BloodTypeFilter> createState() => _BloodTypeFilterState(); }

class _BloodTypeFilterState extends State<_BloodTypeFilter> { String? selectedBloodType;

@override Widget build(BuildContext context) { return Container(height: 50,

padding: const EdgeInsets.symmetric(horizontal: AppSpacing.md), child: ListView(

scrollDirection: Axis.horizontal, children: [_FilterChip(label: 'All',

```
isSelected: selectedBloodType == null,

onTap: () => setState(() => selectedBloodType = null), ), ...BloodType.all.map((type) => _FilterChip( label: type,

isSelected: selectedBloodType == type,

onTap: () => setState(() => selectedBloodType = type), )), ], ),

); } }

class _FilterChip extends StatelessWidget { final String label; final bool isSelected; final VoidCallback onTap; const
_FilterChip({
```

```
required this.label, required this.isSelected, required this.onTap, });

@override Widget build(BuildContext context) { return Padding(

padding: const EdgeInsets.only(right: AppSpacing.sm), child: FilterChip( label: Text(label), selected: isSelected,
onSelected: (_) => onTap(),

selectedColor: AppColors.primary.withValues(alpha: 0.2), checkmarkColor: AppColors.primary, ),

); } }

class _BloodRequestCard extends StatelessWidget { final BloodRequest request;

const _BloodRequestCard({required this.request}); @override Widget build(BuildContext context) { return Card(

margin: const EdgeInsets.only(bottom: AppSpacing.md), child: Padding(

padding: const EdgeInsets.all(AppSpacing.md), child: Column(

crossAxisAlignment: CrossAxisAlignment.start, children: [ Row( children: [ Container(

padding: const EdgeInsets.symmetric(horizontal: AppSpacing.sm, vertical: AppSpacing.xs), decoration: BoxDecoration(
color: AppColors.primary,

borderRadius: BorderRadius.circular(AppRadius.sm), ), child: Text( request.bloodType,

style: const TextStyle(color: Colors.white, fontWeight: FontWeight.bold),

),
```

```
),
const SizedBox(width: AppSpacing.sm), if (request.isUrgent) Container(
padding: const EdgeInsets.symmetric(horizontal: AppSpacing.sm, vertical: AppSpacing.xs), decoration: BoxDecoration(
color: AppColors.error,
borderRadius: BorderRadius.circular(AppRadius.sm), ),
child: const Row(
mainAxisSize: MainAxisSize.min, children: [ Icon(Icons.warning, size: 14, color: Colors.white),
SizedBox(width: 4), Text('URGENT', style: TextStyle(color: Colors.white, fontSize: 12)), ], ), ),
const Spacer(), Text( '${request.unitsNeeded} units',
style: Theme.of(context).textTheme.titleMedium?.copyWith(fontWeight: FontWeight.bold), ), ], ),
const SizedBox(height: AppSpacing.md), Row( children: [
const Icon(Icons.location_on, size: 16, color: Colors.grey),
```

```
const SizedBox(width: AppSpacing.xs), Text(request.location, style: Theme.of(context).textTheme.bodyMedium), ], ),
const SizedBox(height: AppSpacing.sm), Row( children: [
const Icon(Icons.person, size: 16, color: Colors.grey), const SizedBox(width: AppSpacing.xs),
Text(request.receiverName, style: Theme.of(context).textTheme.bodyMedium), ], ),
const SizedBox(height: AppSpacing.md), Row( children: [ Expanded(
child: OutlinedButton( onPressed: () {},
child: const Text('View Details'), ), ),
const SizedBox(width: AppSpacing.md), Expanded(
child: ElevatedButton( onPressed: () {},
child: const Text('Pledge'), ), ), ], ), ],
```

```
), ),

); } }

class _EmptyState extends StatelessWidget { @override Widget build(BuildContext context) { return Center( child: Column(

mainAxisAlignment: MainAxisAlignment.center, children: [ Icon(Icons.inbox, size: 64, color: Colors.grey.shade400), const SizedBox(height: AppSpacing.md), Text( 'No blood requests',

style: Theme.of(context).textTheme.bodyLarge?.copyWith(color: Colors.grey), ), ], ),

); } }

class _LoadingState extends StatelessWidget { @override Widget build(BuildContext context) { return ListView.builder(

padding: const EdgeInsets.all(AppSpacing.md), itemCount: 5,

itemBuilder: (context, index) { return Shimmer.fromColors( baseColor: Colors.grey.shade300, highlightColor: Colors.grey.shade100, child: Card(

margin: const EdgeInsets.only(bottom: AppSpacing.md), child: Container( height: 150,

decoration: BoxDecoration( color: Colors.white,

borderRadius: BorderRadius.circular(AppRadius.md),
```

```
), ), ), ); },

); } }

class _ErrorState extends StatelessWidget { final String message;

const _ErrorState({required this.message}); @override Widget build(BuildContext context) { return Center( child: Column(

mainAxisAlignment: MainAxisAlignment.center, children: [ Icon(Icons.error_outline, size: 64, color: AppColors.error), const SizedBox(height: AppSpacing.md),

Text(
```

```
'Something went wrong',

style: Theme.of(context).textTheme.bodyLarge, ),

const SizedBox(height: AppSpacing.md), ElevatedButton( onPressed: () {}, child: const Text('Retry'), ), ], ),

); } }
```

5.2.9. blood_request_detail_screen.dart

Path: features/blood_requests/presentation/screens/blood_request_detail_screen.dart | Lines (stripped): 257

```
import 'package:flutter/material.dart';

import 'package:flutter_riverpod/flutter_riverpod.dart'; import '../../../../core/constants/app_colors.dart'; import
'../../../../core/constants/app_spacing.dart'; import '../../../../core/constants/app_radius.dart';

import '../../../blood_requests/data/blood_request_provider.dart'; import
'../../../../shared/services/blood_request_model.dart'; import '../../../auth/presentation/providers/local_auth_provider.dart';
class BloodRequestDetailScreen extends ConsumerWidget { final String requestId;

const BloodRequestDetailScreen({super.key, required this.requestId}); @override Widget build(BuildContext context,
WidgetRef ref) {

final requestAsync = ref.watch(bloodRequestDetailProvider(requestId)); final authState =
ref.watch(localAuthStateProvider); final currentUserId = authState.valueOrNull?.uid; return Scaffold(

appBar: AppBar(title: const Text('Blood Request')), body: requestAsync.when(

data: (request) => _RequestDetailContent(request: request, currentUserId: currentUserId), loading: () => const
Center(child: CircularProgressIndicator()), error: (e, _) => Center(child: Text('Error: $e')), ),

); } }

class _RequestDetailContent extends ConsumerWidget { final BloodRequest request; final String? currentUserId;

const _RequestDetailContent({required this.request, this.currentUserId}); @override Widget build(BuildContext context,
WidgetRef ref) { final isOwner = request.receiverId == currentUserId;

final hasPledged = request.pledgedDonors.contains(currentUserId);
```

```
child: Text(label, style: TextStyle(color: color, fontSize: 12, fontWeight: FontWeight.w600)), ); } }
```

```
class _InfoCard extends StatelessWidget { final BloodRequest request;

const _InfoCard({required this.request}); @override Widget build(BuildContext context) { return Card( child: Padding(

padding: const EdgeInsets.all(AppSpacing.md), child: Column(

crossAxisAlignment: CrossAxisAlignment.start, children: [ _InfoRow(icon: Icons.person, label: 'Patient', value:
request.receiverName), const Divider(), _InfoRow(icon: Icons.location_on, label: 'Location', value: request.location), if
(request.hospitalName != null) ...[ const Divider(), _InfoRow(icon: Icons.local_hospital, label: 'Hospital', value:
request.hospitalName!), ],

if (request.contactPhone != null) ...[ const Divider(), _InfoRow(icon: Icons.phone, label: 'Contact', value:
request.contactPhone!), ],

const Divider(), _InfoRow(icon: Icons.calendar_today, label: 'Posted', value: _formatDate(request.createdAt)), ], ), ),

);

} String _formatDate(DateTime dt) => '${dt.day}/${dt.month}/${dt.year}'; }

class _InfoRow extends StatelessWidget { final IconData icon; final String label; final String value;

const _InfoRow({required this.icon, required this.label, required this.value}); @override Widget build(BuildContext
context) { return Padding(

padding: const EdgeInsets.symmetric(vertical: AppSpacing.sm), child: Row( children: [ Icon(icon, size: 20, color:
Colors.grey), const SizedBox(width: AppSpacing.md), Column(

crossAxisAlignment: CrossAxisAlignment.start, children: [ Text(label, style:
Theme.of(context).textTheme.bodySmall?.copyWith(color: Colors.grey)), Text(value, style:
Theme.of(context).textTheme.bodyLarge), ], ), ], ),

); } }

class _DonorsCard extends StatelessWidget { final BloodRequest request;

const _DonorsCard({required this.request});

@override
```

```
Widget build(BuildContext context) { return Card( child: Padding(

padding: const EdgeInsets.all(AppSpacing.md), child: Column(

crossAxisAlignment: CrossAxisAlignment.start, children: [
```

```
Row( children: [

const Icon(Icons.people, color: AppColors.primary), const SizedBox(width: AppSpacing.sm), Text('Pledged Donors',
style: Theme.of(context).textTheme.titleMedium), ], ),

const SizedBox(height: AppSpacing.md), if (request.pledgedDonors.isEmpty)

const Text('No donors have pledged yet', style: TextStyle(color: Colors.grey)) else
...request.pledgedDonors.asMap().entries.map((entry) => ListTile(

leading: CircleAvatar(backgroundColor: AppColors.success.withValues(alpha: 0.1), child: co nst Icon(Icons.person,
color: AppColors.success)), title: Text('Donor ${entry.key + 1}'), subtitle: const Text('Pledged to donate'),

trailing: const Icon(Icons.check_circle, color: AppColors.success), )), ], ), ),

); } }
```

5.2.10. post_blood_request_screen.dart

Path: features/blood_requests/presentation/screens/post_blood_request_screen.dart | Lines (stripped): 204

```
import 'package:flutter/material.dart';

import 'package:flutter_riverpod/flutter_riverpod.dart'; import 'package:go_router/go_router.dart'; import
'../../../../core/constants/app_colors.dart'; import '../../../../core/constants/app_spacing.dart'; import
'../../../../shared/services/blood_request_model.dart'; import '../../../../shared/services/local_blood_request_service.dart';
import 'package:flutter_secure_storage/flutter_secure_storage.dart';

final postBloodRequestProvider = StateNotifierProvider<PostBloodRequestNotifier, PostBloodRequestState>((ref) {

return PostBloodRequestNotifier(ref); });

class PostBloodRequestState { final bool isLoading; final String? error;

const PostBloodRequestState({this.isLoading = false, this.error}); PostBloodRequestState copyWith({bool? isLoading,
String? error}) { return PostBloodRequestState( isLoading: isLoading ?? this.isLoading, error: error, ); } }

class PostBloodRequestNotifier extends StateNotifier<PostBloodRequestState> { final Ref _ref;

final FlutterSecureStorage _secureStorage = const FlutterSecureStorage(); PostBloodRequestNotifier(this._ref) :
super(const PostBloodRequestState()); Future<void> submitRequest({ required String bloodType, required int units,
required String location, required String urgency, required String hospitalName, required String contactPhone, }) async {
try {

state = state.copyWith(isLoading: true);

final uid = await _secureStorage.read(key: 'user_id'); if (uid == null) {

state = state.copyWith(isLoading: false, error: 'User not logged in'); return; }

final requestService = _ref.read(localBloodRequestServiceProvider);
```

```
await requestService.createRequest( receiverId: uid, receiverName: 'User', bloodType: bloodType, unitsNeeded: units,
location: location, urgency: urgency, hospitalName: hospitalName, contactPhone: contactPhone, );

state = state.copyWith(isLoading: false); } catch (e) {

state = state.copyWith(isLoading: false, error: e.toString()); } } }

class PostBloodRequestScreen extends ConsumerStatefulWidget {
```

```
const PostBloodRequestScreen({super.key}); @override ConsumerState<PostBloodRequestScreen> createState() =>
_PostBloodRequestScreenState(); }

class _PostBloodRequestScreenState extends ConsumerState<PostBloodRequestScreen> { final _formKey =
GlobalKey<FormState>();

final _locationController = TextEditingController(); final _hospitalController = TextEditingController(); final
_phoneController = TextEditingController(); String? _selectedBloodType; int _units = 1; String _urgency = 'normal';
@override void dispose() { _locationController.dispose();

_hospitalController.dispose(); _phoneController.dispose(); super.dispose(); } Future<void> _submit() async {

if (!_formKey.currentState!.validate()) return; if (_selectedBloodType == null) {
ScaffoldMessenger.of(context).showSnackBar(

const SnackBar(content: Text('Please select blood type')), ); return; }

await ref.read(postBloodRequestProvider.notifier).submitRequest( bloodType: _selectedBloodType!, units: _units,

location: _locationController.text, urgency: _urgency,

hospitalName: _hospitalController.text, contactPhone: _phoneController.text, );

if (mounted) { context.pop(); } }

@override Widget build(BuildContext context) {

final state = ref.watch(postBloodRequestProvider); return Scaffold( appBar: AppBar(

title: const Text('Post Blood Request'), ),

body: SafeArea(

child: SingleChildScrollView(

padding: const EdgeInsets.all(AppSpacing.lg), child: Form( key: _formKey, child: Column(

crossAxisAlignment: CrossAxisAlignment.stretch, children: [
```

```
Text('Blood Type Required', style: Theme.of(context).textTheme.titleSmall), const SizedBox(height: AppSpacing.sm),
Wrap(

spacing: AppSpacing.sm,

children: BloodType.all.map((type) { final isSelected = _selectedBloodType == type;
```

```
return ChoiceChip( label: Text(type), selected: isSelected,

onSelected: (_) => setState(() => _selectedBloodType = type), selectedColor: AppColors.primary.withValues(alpha: 0.2),
); }).toList(), ),

const SizedBox(height: AppSpacing.lg), Text('Units Needed', style: Theme.of(context).textTheme.titleSmall), const
SizedBox(height: AppSpacing.sm), Row( children: [ IconButton(

onPressed: _units > 1 ? () => setState(() => _units--) : null, icon: const Icon(Icons.remove_circle_outline), ),
Text('$_units', style: Theme.of(context).textTheme.headlineSmall), IconButton(

onPressed: _units < 10 ? () => setState(() => _units++) : null, icon: const Icon(Icons.add_circle_outline), ), ], ),

const SizedBox(height: AppSpacing.lg), Text('Urgency', style: Theme.of(context).textTheme.titleSmall), const
SizedBox(height: AppSpacing.sm), SegmentedButton<String>( segments: const [ ButtonSegment(value: 'normal', label:
Text('Normal')), ButtonSegment(value: 'urgent', label: Text('Urgent')), ],

selected: {_urgency},

onSelectionChanged: (value) => setState(() => _urgency = value.first), ),

const SizedBox(height: AppSpacing.lg), TextFormField(

controller: _locationController, decoration: const InputDecoration( labelText: 'Location', hintText: 'City/Area',

prefixIcon: Icon(Icons.location_on), ),

validator: (value) => value?.isEmpty ?? true ? 'Required' : null, ),

const SizedBox(height: AppSpacing.md), TextFormField(

controller: _hospitalController, decoration: const InputDecoration( labelText: 'Hospital Name', hintText: 'Enter hospital
name', prefixIcon: Icon(Icons.local_hospital), ),

validator: (value) => value?.isEmpty ?? true ? 'Required' : null, ),

const SizedBox(height: AppSpacing.md), TextFormField(

controller: _phoneController, keyboardType: TextInputType.phone, decoration: const InputDecoration(
```

```
labelText: 'Contact Phone', hintText: '+91 9876543210', prefixIcon: Icon(Icons.phone), ),

validator: (value) => value?.isEmpty ?? true ? 'Required' : null,
```

```
),
const SizedBox(height: AppSpacing.xl), ElevatedButton(
onPressed: state.isLoading ? null : _submit, child: state.isLoading ? const SizedBox( height: 20, width: 20,
child: CircularProgressIndicator(strokeWidth: 2), ) : const Text('Post Request'), ), ], ), ), ), ),
); } }
```

5.2.11. profile_screen.dart

Path: features/profile/presentation/screens/profile_screen.dart | Lines (stripped): 200

```
import 'package:flutter/material.dart';

import 'package:flutter_riverpod/flutter_riverpod.dart'; import 'package:go_router/go_router.dart'; import
'../../../../core/constants/app_colors.dart'; import '../../../../core/constants/app_spacing.dart'; import
'../../../../core/constants/app_radius.dart'; import '../../../../core/router/app_routes.dart'; import
'../../../../shared/services/user_model.dart';

import '../../../auth/presentation/providers/local_auth_provider.dart'; class ProfileScreen extends ConsumerWidget { const
ProfileScreen({super.key}); @override Widget build(BuildContext context, WidgetRef ref) { final authState =
ref.watch(localAuthStateProvider); final user = authState.valueOrNull; return Scaffold( appBar: AppBar(

title: const Text('Profile'), actions: [ IconButton(

icon: const Icon(Icons.edit),

onPressed: () => context.push(AppRoutes.editProfile), ), ], ),

body: SafeArea(

child: SingleChildScrollView(

padding: const EdgeInsets.all(AppSpacing.lg), child: Column( children: [ _ProfileHeader(user: user),

const SizedBox(height: AppSpacing.lg), _ProfileInfo(user: user),

const SizedBox(height: AppSpacing.lg), _VerificationStatus(status: user?.verificationStatus ??
VerificationStatus.pending), const SizedBox(height: AppSpacing.lg), _MenuItems(), ], ), ), ),

); } }

class _ProfileHeader extends StatelessWidget { final User? user;

const _ProfileHeader({this.user});
```

```
),
const SizedBox(height: AppSpacing.xs), Text(
user?.phone ?? '',
style: Theme.of(context).textTheme.bodyMedium?.copyWith(color: Colors.grey), ),
const SizedBox(height: AppSpacing.xs), Container(
padding: const EdgeInsets.symmetric(horizontal: AppSpacing.md, vertical: AppSpacing.xs), decoration: BoxDecoration(
color: AppColors.secondary.withValues(alpha: 0.1), borderRadius: BorderRadius.circular(AppRadius.full), ), child: Text(
user?.role.displayName ?? '',
style: TextStyle(color: AppColors.secondary, fontWeight: FontWeight.w500), ), ), ],
); } }
class _ProfileInfo extends StatelessWidget { final User? user;
const _ProfileInfo({this.user}); @override Widget build(BuildContext context) { return Card( child: Padding(
padding: const EdgeInsets.all(AppSpacing.md), child: Column( children: [ _InfoRow(label: 'Blood Type', value:
user?.bloodType ?? 'Not set'), const Divider(), _InfoRow(label: 'Member Since', value: user?.createdAt.toString().split('
')[0] ?? ''), ], ), ),
); } }
class _InfoRow extends StatelessWidget { final String label; final String value;
const _InfoRow({required this.label, required this.value}); @override Widget build(BuildContext context) { return
Padding(
padding: const EdgeInsets.symmetric(vertical: AppSpacing.sm), child: Row(
mainAxisAlignment: MainAxisAlignment.spaceBetween, children: [ Text(label, style:
Theme.of(context).textTheme.bodyMedium), Text(value, style:
Theme.of(context).textTheme.bodyMedium?.copyWith(fontWeight: FontWeight.w500)), ], ),
); }
}
class _VerificationStatus extends StatelessWidget {
```

```
final VerificationStatus status;

const _VerificationStatus({required this.status}); @override Widget build(BuildContext context) { Color color;

IconData icon; String label; switch (status) { case VerificationStatus.verified:

color = AppColors.success; icon = Icons.verified; label = 'Verified'; break; case VerificationStatus.pending: case
VerificationStatus.inReview:

color = AppColors.warning; icon = Icons.pending; label = 'Pending Verification'; break; case VerificationStatus.rejected:

color = AppColors.error; icon = Icons.cancel;

label = 'Verification Rejected'; break; }

return Card(

color: color.withValues(alpha: 0.1), child: Padding(

padding: const EdgeInsets.all(AppSpacing.md), child: Row( children: [ Icon(icon, color: color),

const SizedBox(width: AppSpacing.md), Expanded( child: Text( label,

style: TextStyle(color: color, fontWeight: FontWeight.w500), ), ), ], ), ),

); } }

class _MenuItems extends ConsumerWidget { @override Widget build(BuildContext context, WidgetRef ref) { return
Card( child: Column( children: [ ListTile(

leading: const Icon(Icons.settings), title: const Text('Settings'),

trailing: const Icon(Icons.chevron_right), onTap: () {}, ),

const Divider(height: 1), ListTile(

leading: const Icon(Icons.help),
```

```
title: const Text('Help & Support'), trailing: const Icon(Icons.chevron_right), onTap: () {}, ),
const Divider(height: 1), ListTile(
leading: Icon(Icons.logout, color: AppColors.error),
title: Text('Logout', style: TextStyle(color: AppColors.error)), onTap: () async {
```

```
await ref.read(localAuthStateProvider.notifier).signOut(); if (context.mounted) { context.go(AppRoutes.login); }
}, ), ], ),
); } }
```

5.2.12. notifications_screen.dart

Path: features/notifications/presentation/screens/notifications_screen.dart | Lines (stripped): 117

```
import 'package:flutter/material.dart';

import 'package:flutter_riverpod/flutter_riverpod.dart'; import '../../../../core/constants/app_colors.dart'; import '../../../../core/constants/app_spacing.dart';

final notificationsProvider = FutureProvider<List<NotificationItem>>((ref) async { return []; });

class NotificationItem { final String id; final String title; final String body; final DateTime createdAt; final bool isRead; NotificationItem({ required this.id, required this.title, required this.body, required this.createdAt, this.isRead = false, }); }

class NotificationsScreen extends ConsumerWidget { const NotificationsScreen({super.key}); @override Widget build(BuildContext context, WidgetRef ref) { final notificationsAsync = ref.watch(notificationsProvider); return Scaffold( appBar: AppBar(

title: const Text('Notifications'), ),

body: SafeArea(

child: notificationsAsync.when(

data: (notifications) => notifications.isEmpty ? Center( child: Column(

mainAxisAlignment: MainAxisAlignment.center, children: [ Icon(Icons.notifications_none, size: 64, color: Colors.grey.shade400), const SizedBox(height: AppSpacing.md), Text( 'No notifications',

style: Theme.of(context).textTheme.bodyLarge?.copyWith(color: Colors.grey), ), ],

),

) : ListView.builder(

padding: const EdgeInsets.all(AppSpacing.md), itemCount: notifications.length, itemBuilder: (context, index) {

return _NotificationCard(notification: notifications[index]); }, ),

loading: () => const Center(child: CircularProgressIndicator()), error: (error, stack) => Center( child: Column(
```

mainAxisAlignment: MainAxisAlignment.center, children: [Icon(Icons.error_outline, size: 64, color: AppColors.error), const SizedBox(height: AppSpacing.md),

```
const Text('Something went wrong'), const SizedBox(height: AppSpacing.md), ElevatedButton(

onPressed: () => ref.refresh(notificationsProvider), child: const Text('Retry'), ), ], ), ), ), ),

); } }

class _NotificationCard extends StatelessWidget { final NotificationItem notification;

const _NotificationCard({required this.notification}); @override Widget build(BuildContext context) { return Card(

margin: const EdgeInsets.only(bottom: AppSpacing.sm), child: ListTile( leading: Container(

padding: const EdgeInsets.all(AppSpacing.sm), decoration: BoxDecoration(

color: notification.isRead ? Colors.transparent : AppColors.primary.withValues(alpha: 0.1), shape: BoxShape.circle, ),
child: Icon( Icons.notifications,

color: notification.isRead ? Colors.grey : AppColors.primary, ), ), title: Text( notification.title,

style: TextStyle(fontWeight: notification.isRead ? FontWeight.normal : FontWeight.bold), ),

subtitle: Text(notification.body), trailing: Text( _formatTime(notification.createdAt), style:
Theme.of(context).textTheme.bodySmall, ), ),

);

} String _formatTime(DateTime time) { final now = DateTime.now(); final diff = now.difference(time); if (diff.inMinutes
< 60) { return '${diff.inMinutes}m ago'; } else if (diff.inHours < 24) { return '${diff.inHours}h ago'; } else {

return '${diff.inDays}d ago'; } } }
```

5.3. RBAC

5.3.1. rbac_service.dart

Path: rbac/rbac_service.dart | Lines (stripped): 25

```
import 'package:flutter_riverpod/flutter_riverpod.dart'; import 'models/app_role.dart'; import
'models/app_permission.dart'; import 'role_permission_matrix.dart';

final rbacServiceProvider = Provider<RbacService>((ref) => RbacService()); class RbacService {

bool hasPermission(AppRole role, AppPermission permission) { return RolePermissionMatrix.hasPermission(role,
permission); }

bool hasAnyPermission(AppRole role, Set<AppPermission> permissions) { return
RolePermissionMatrix.hasAnyPermission(role, permissions); }

bool hasAllPermissions(AppRole role, Set<AppPermission> permissions) { return
RolePermissionMatrix.hasAllPermissions(role, permissions); } Set<AppPermission> getPermissionsForRole(AppRole
role) { return RolePermissionMatrix.getPermissionsForRole(role); }

bool isVerifiedUser(AppRole role, bool isVerified) {

if (role == AppRole.superAdmin || role == AppRole.admin || role == AppRole.verifier) { return true; }

return isVerified; } }
```

5.4. SHARED

5.4.1 user_model.dart

Path: shared/services/user_model.dart | Lines (stripped): 117

```
import 'package:equatable/equatable.dart'; import '../../rbac/models/app_role.dart'; enum VerificationStatus { pending, inReview, verified, rejected; String get displayName { switch (this) { case VerificationStatus.pending:

return 'Pending'; case VerificationStatus.inReview:

return 'In Review'; case VerificationStatus.verified:

return 'Verified'; case VerificationStatus.rejected:

return 'Rejected'; } }

static VerificationStatus fromString(String value) { return VerificationStatus.values.firstWhere( (status) => status.name == value, orElse: () => VerificationStatus.pending, ); } }

class User extends Equatable { final String uid; final String phone; final AppRole role;

final VerificationStatus verificationStatus; final DateTime createdAt; final DateTime updatedAt; final String? bloodType; final String? name; final bool isAvailable; final bool isBanned; const User({
```

```
createdAt, updatedAt, bloodType, name, isAvailable, isBanned, ]; }
```

CHAPTER – VI

SYSTEM TESTING

6.0. SYSTEM TESTING

System Testing

System testing is an important phase in the development of the Blood Donation System. It is performed to ensure that the entire application works correctly and meets the required specifications. The purpose of system testing is to identify errors, verify the functionality of the system, and ensure that all modules work together properly.

During system testing, the complete application is tested as a whole rather than testing individual components separately. This process helps confirm that the integration between different modules such as the **Donor Module, Blood Bank Module, Chat Module, and Search Module** is functioning correctly.

6.1. Unit Testing

Unit testing is performed to test individual modules of the system independently.

6.1.1 Authentication Module Testing

Authentication Module Testing

Authentication Module Testing is performed to verify that the login and registration features of the Blood Donation System work correctly and securely. The purpose of this testing is to ensure that only authorized users can access the system and that user information is handled safely.

In this module, users such as donors, blood bank staff, or patients must first register by providing their details such as name, email, password, and contact information. After successful registration, users can log in to the application using their credentials. Authentication testing checks whether the system correctly verifies the user's identity before granting access.

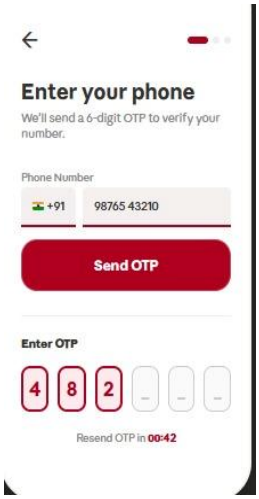
During testing, different scenarios are examined. For example, the system is tested to confirm that users can successfully register with valid information. It also verifies that users can log in only with the correct username and password. If incorrect login details are entered, the system should display an appropriate error message and prevent unauthorized access.

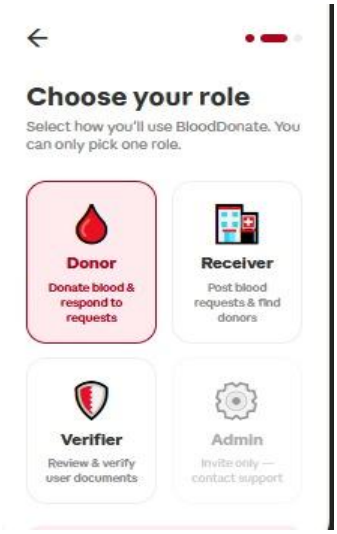
The testing process also checks whether the system properly stores user credentials in the database and retrieves them during login verification. Input validation is tested to ensure that invalid or incomplete data cannot be submitted during registration.

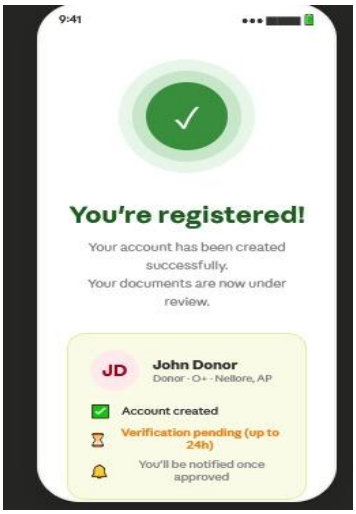
Additionally, security measures are tested to ensure that user data is protected and that the authentication process works reliably. This helps maintain the privacy and security of donor and user information.

Overall, Authentication Module Testing ensures that the login and registration system of the Blood Donation System functions correctly, provides secure access, and protects the system from unauthorized users.

Result: Authentication module functions correctly.







6.1.2. Item Management Testing

Item Management Testing is performed to ensure that the system correctly handles the management of items or records within the Blood Donation System. In this project, items mainly refer to blood-related records such as blood group details, donor information, and blood stock data maintained by the blood bank.

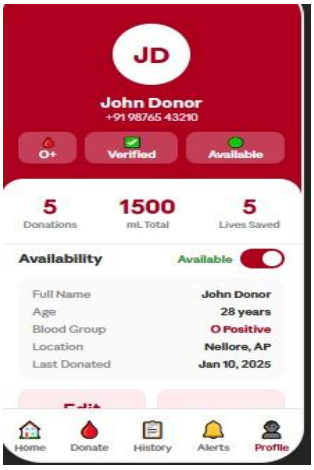
The purpose of this testing is to verify that users such as administrators or blood bank staff can successfully add, update, view, and delete records in the system. The testing process ensures that the item management functions work correctly and that the information stored in the database is accurate and properly organized.

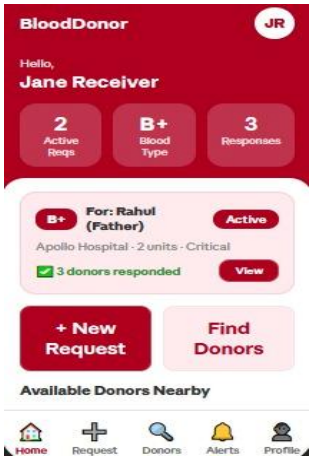
During testing, different scenarios are checked. For example, the system is tested to confirm that new blood stock records can be added successfully with correct details such as blood group, number of units available, and blood bank information. The update function is tested to ensure that existing records can be modified whenever there is a change in blood availability.

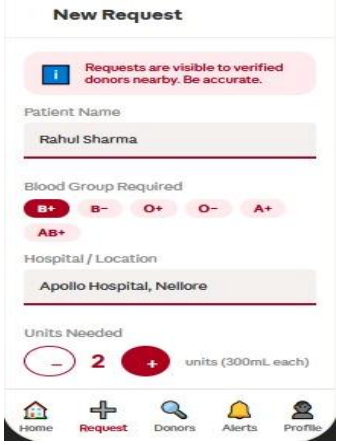
The delete function is also tested to confirm that records can be removed when they are no longer required. Additionally, the system checks whether the correct information is displayed when users view or search for blood stock details.

The testing process also verifies that invalid or incomplete data cannot be entered into the system. Input validation ensures that only proper and required information is accepted and stored in the database.

Overall, Item Management Testing ensures that the system can efficiently manage blood-related records and maintain accurate information, which is essential for the smooth operation of the Blood Donation System.







6.2. Integration Testing

Integration testing ensures that different modules work together properly.

Tested interactions between:

- Login → Home Screen
- Item Upload → Item Display
- Item Click → Chat Opening
- Chat → Inbox Update
- Mark as Donated → Chat Restriction

Result: All integrated modules function smoothly without conflict.

6.3. System Testing

System testing evaluates the entire application as a complete system.

- Verified overall application stability.
- Tested performance under multiple operations.
- Checked data consistency in Firestore.
- Verified secure data access through Firestore rules.

Result: The system operates reliably and securely.

6.4. User Acceptance Testing (UAT)

The application was tested by students to evaluate usability and functionality.

Feedback included:

- Easy navigation
- Clear user interface
- Fast real-time chat
- Efficient item search

The system was accepted as user-friendly and effective.

6.5. Security Testing

Security testing was conducted to ensure:

- Unauthorized users cannot access data.
- Chat is restricted to participants.
- Users cannot modify other users' items.
- Messages are permanently stored.

Result: Security rules function correctly and prevent unauthorized access.

6.6 Conclusion of Testing

All modules of the College Marketplace system were tested successfully. The system meets functional and non-functional requirements and operates efficiently, securely, and reliably.

CHAPTER VII

CONCLUSION

Conclusion

The **Blood Donation System** is developed to improve the management of blood donation and blood distribution by using modern mobile technology. Blood is one of the most important and life-saving resources in the healthcare sector. It is required for many medical situations such as surgeries, accident treatments, cancer therapy, organ transplants, and maternity care. Since blood cannot be artificially produced, hospitals and healthcare centers depend completely on voluntary blood donors and well-maintained blood banks. Therefore, it is very important to have an efficient system that can help manage blood donation and ensure the availability of blood whenever it is required.

Traditional blood management systems often rely on manual processes such as maintaining records in registers or spreadsheets. These methods are time-consuming, difficult to manage, and sometimes lead to errors or outdated information. In emergency situations, patients or their family members may struggle to find the required blood group quickly. They may need to contact several hospitals or blood banks, which can cause delays and create stressful situations. These limitations highlight the need for a modern and efficient solution to manage blood donation activities.

The Blood Donation System addresses these problems by providing a digital platform that connects donors, blood banks, and patients through a mobile application. The system simplifies the process of blood donation and makes it easier for users to find blood availability when needed. Donors can register in the application by providing their personal information, blood group, and availability status. This information is stored in a centralized database, making it easier for the system to identify and contact suitable donors when there is a blood request.

Blood banks can also use the system to manage and update blood stock information in real time. They can record the number of blood units available for different blood groups and update the data whenever new donations are received or blood is used for patients. This helps maintain accurate records and ensures that blood availability information is always up to date.

Patients or their relatives can use the application to search for the required blood group and find nearby donors or blood banks. The search and filtering feature allows users to quickly identify the most suitable options based on blood group and availability. In addition, the chat module allows users to communicate directly with donors or blood banks, which helps improve coordination and reduce the time required to arrange blood during emergencies.

The application is developed using **Dart programming language** and supported by modern development tools such as **Gravity AI**. These technologies help in creating a fast, efficient, and user-friendly mobile application. The system also includes features such as secure authentication, database integration, and performance optimization to ensure that the application operates smoothly and safely.

Another important benefit of the Blood Donation System is that it helps promote awareness about voluntary blood donation. Many people are willing to donate blood but may not know where or when their help is needed. By providing a platform that connects donors with patients and blood banks, the system encourages more individuals to participate in blood donation activities. This can help increase the availability of blood and support healthcare services in saving lives.

The testing and evaluation of the system ensure that all modules work correctly and that the application is reliable for real-world use. Different types of testing, including authentication testing, item management testing, and system testing, are carried out to verify that the system functions properly and securely.

In conclusion, the Blood Donation System is a practical and effective solution for improving blood donation management. It reduces the difficulties associated with traditional manual systems and provides a faster and more

organized way to connect donors, blood banks, and patients. By using modern mobile technology, the system improves communication, increases efficiency, and ensures quick access to blood availability information.

Most importantly, the system contributes to saving human lives by helping patients receive the blood they need at the right time. With further improvements and wider adoption, the Blood Donation System can play a significant role in strengthening healthcare services and promoting the spirit of voluntary blood donation in society.

CHAPTER VIII

REFERENCES

REFERENCES

References

1. World Health Organization. *Blood Safety and Availability*.
   Available at: https://www.who.int
   (Information about global blood donation systems and blood safety guidelines.)

2. Indian Red Cross Society. *Blood Donation and Blood Bank Management*.
   Available at: https://www.indianredcross.org

3. National Health Portal of India. *Voluntary Blood Donation Awareness*.
   Available at: https://www.nhp.gov.in

4. Software Engineering, by Ian Sommerville.
   Pearson Education, 10th Edition.
   (Used for understanding system development methodologies.)

5. Android Programming: The Big Nerd Ranch Guide by Bill Phillips and Chris Stewart.
   Big Nerd Ranch Guides.
   (Reference for Android application development.)

6. Android Studio Official Documentation.
   Available at: https://developer.android.com

7. Firebase Documentation.
   Available at: https://firebase.google.com/docs
   (Used for database integration and authentication.)